Mercor Execution Plan: exactly what to do, Thu 09-03 to Wed 09-09 13:00 ET

Companion to `mercor-research-engineer-post-training.md` (the intel doc). This file is the to-do list. Every task has a time, exact steps, a written output, and a done-check. Study material lives at the cited source; each study unit says what to read, what to learn from it, a plain-language version, and self-check questions with an answer key.

Total work in the compressed schedule: about 4 hours. §1 says what to cut and in what order.

---

0. The target

The interviewer (James Moore, research eng, 20 minutes on Google Meet) will write a scorecard afterward. You win if he can write these three lines without hedging:

1. **"Has actually done rigorous failure analysis and reward/eval design, with numbers."** (BMO bias pipeline; proprioceptive collapse; LIBERO-PRO recipe result.)
2. **"Knows what our team does and why; read the 397B write-up and understood it."**
3. **"Honest about the gap (no LLM-scale RL runs), and the gap is learnable."**

Everything below exists to make those three lines easy for him to write. Anything that does not serve one of them gets cut.

Round facts (from Aksh's 08-28 email): "a 20 minute conversation with a Hiring Manager. This will be a technical conversation about your background & skillset. You'll also get an opportunity to learn more about the team." Not a coding round. Not a recruiter gate.

Time budget inside the call: 0:00–2:00 intro and your pitch; 2:00–14:00 his questions (expect 4 to 6, with follow-ups); 14:00–18:00 your questions; 18:00–20:00 close. Every answer you give must have a version under 90 seconds. If he wants more, he will ask.

---

1. Schedule (compressed: Mon 09-07 night to Wed 09-09 13:00 ET)

Real clock as of Monday night: about 30 min tonight, a few hours Tuesday around classes and work, Wednesday morning around classes and work. Total: about 4 hours. Everything below is ordered by value; do the tasks in order and stop when the time is gone.

When	Minutes	Task	Output
Mon night	15	T1 Decisions D1, D2, D3 only (§4.1). D1 is the start-date sentence; write it.	Three sentences written
Mon night	15	T2 Pitch: read §2.1 twice, then record it on your phone, timed. Two takes. Under 90 s.	Recording exists
Tue, block 1	45	T3 Take-home recall (§4.2)	Worksheet items 1, 2, 5, 6, 7 written; said aloud once

When	Minutes	Task	Output
Tue, block 2	60	T4 Unit A: read the 397B post only (skip the README), then Self-check A from memory	Self-check A done; misses re-read
Tue, block 3	30	T5 Unit B, short form: read the four plain versions in §3.2 (B1, B2, B3, B4) and B5; read DeepSeekMath §4.1 only if there is time. Answer the B self-checks from the plain versions.	B self-checks answered
Tue, block 4	15	T8 Numbers card (§5), cover-and-recite, two passes	100% on second pass
Tue, any gap	10	Unit C plain version only (§3.3), once. T9: pick your 3 questions from §2.9 (defaults are fine).	Questions written on paper
Wed morning	30	T10 Practice, short form: run the §6 invocation with questions 1, 2, 3, 4, 6 only. Out loud, timed. Fix any 🔴 once.	Session summary saved
Wed morning	10	T11 Meet test: join the link from the laptop and desk you will use; camera, headset, background. Dial-in on paper.	Done
Wed 11:30 on	see §7	Day-of run sheet	Done

Cut list if Tuesday shrinks, in this order: Unit C, DeepSeekMath reading, T8 second pass, T10 (replace with reading §2 aloud once, timed). Never cut T1, T2, T3, T4.

Stories (§4.3) are not scheduled. You have three stories in §2.4, §2.5 and the pitch; that covers a 20-minute screen. Fill §4.3 only if a Tuesday block finishes early.

---|---|---|---| | Thu 09-03, tonight | 45 | T1 Decisions worksheet (§4.1) · T2 Pitch recording | Worksheet filled; a phone recording of the pitch under 90 s | | Fri 09-04 | 60 | T3 Take-home recall worksheet (§4.2) | Worksheet filled, 5-line problem summary written | | Sat 09-05 | 120 | T4 Study Unit A: the 397B write-up + recipe (§3.1) | Self-check A passed from memory; 6-sentence summary written | | Sun 09-06 | 120 | T5 Study Unit B: RL post-training concepts (§3.2) · T6 Study Unit C: APEX-Agents (§3.3) | Self-checks B and C passed from memory | | Mon 09-07 | 75 | T7 Stories fill-in (§4.3) · T8 Numbers card drill (§5) · T9 Choose your 3 questions | Three STAR+R stories complete; numbers 100% on cover-and-recite; 3 questions written on the day-of sheet | | Tue 09-08 | 75 | T10 Practice session (§6) · T11 Google Meet test | Session summary saved; camera/mic/headset verified; dial-in on paper | | Wed 09-09 | see §7 | Day-of run sheet | Done |

If you miss a day, do not stack. Drop in this order: T6, T11, T8 (only if numbers are already solid). Never drop T1, T3, T4, T10.

---

2. Scripts: say these, in these words

Rehearse from this section, not from the intel doc. Every fact traces to `cv.md` or `article-digest.md`. Do not add anything that is not in those files.

2.1 The pitch (target 75–90 s; hard stop at 90)

> Two threads. By day I'm an AI research engineer at BMO's AI Centre of Excellence. My main result there: I found a systematic bias toward downplaying investment risk in a GenAI tool serving over $200B in wealth-management assets, then built a deterministic eval pipeline over hundreds of synthesized inputs so that class of failure gets caught at scale. Since then I've been building evals for agents that reason over banking and insurance policy, RL environments for wealth-management agents, and a graph-based agent for multi-hop queries over client data.
>
> By night I co-founded Merlyn Labs, three of us doing robotics research. We placed 8th in Stanford's BEHAVIOR-1K Challenge, where I found that masking 60% of proprioception improved task success by up to 48%. We also wrote up why the published π0.5 checkpoint collapses on LIBERO-PRO position-swap: it's the finetuning recipe, not the architecture, and a conservative recipe doubles success from 21% to 42%. I open-sourced a flow-matching VLA integration for RLinf so people can run RL on BEHAVIOR-1K.
>
> The thread through all of it is finding where a model actually fails and building the eval or the training fix that closes it. That's what your post-training and benchmarks work is, so here I am.

Then stop. Silence is his turn.

2.2 Why Mercor / why this team (45 s)

> Aksh described the team as studying capability limits, building benchmarks that probe them, and post-training to prove lift. That's the loop I already run, just at smaller scale. The concrete reason is the 397B SkyRL write-up: two findings stood out. Harness fixes alone moved the 35B model about six points before any RL, and tasks graded by file diff were much harder to overfit than tasks graded on the final response. That's the same lesson as my LIBERO-PRO work, where a published number turned out to be a recipe artifact. I want to work where the data and the verifiers are the product.

2.3 The gap answer (use the moment he asks about GRPO, RLVR, LLM post-training, or "have you trained LLMs") (40 s)

> Straight answer: my RL post-training is on VLAs, not LLMs. I integrated a flow-matching VLA into RLinf so RL runs on BEHAVIOR-1K, and I'm building RL environments for agents at BMO. I have not run GRPO or DAPO on a language model. I've read your recipe closely, and the part that maps directly is the discipline: drive non-model error to zero before training, and don't trust a gain until an ablation isolates it. The trainer and inference plumbing is the part I'd be learning here, and I'd rather say that than pretend.

Do not soften it, do not extend it. Then let him decide whether to go deeper.

2.4 Hard technical trade-off (75 s)

> On LIBERO-PRO I chose a conservative full finetune over LoRA even though LoRA was cheaper and the default. Result: position-swap success went from the published 21% to 42% while matching standard LIBERO. LoRA sat at 15 to 21% at matched hyperparameters. Why: the failure looked like trajectory memorization, so I wanted the update spread across the network rather than a low-rank patch, and I needed to rule out "it's the architecture." What I did: swept the recipe both ways, found the effect is bounded, batch 16 at LR 1e-6 falls back to 26%, and tested a frozen video-diffusion visual prior, which made it worse, 42 to 35. The trade was compute and time for a clean causal story.

2.5 End-to-end project you drove (60 s)

> The BMO bias finding. A GenAI tool serving over $200B in wealth-management assets had a systematic tendency to downplay investment risk. In wealth management that's a compliance problem. Spot checks would never show "systematic," so I built a deterministic pipeline: hundreds of synthesized inputs, same inputs give comparable outputs run to run, quantify the skew. That pipeline is now how misaligned outputs get detected at scale there.

If he asks for detail beyond this, use your §4.1 boundary decision. Say "that's as far as I can go on the internal detail" and stop.

2.6 "How would you design a verifier that's hard to game?" (75 s)

> From what I've actually built: at Merlyn I'm making VLM judges that score rollouts into dense, context-dependent rewards. Principles I'd defend: grade state, not narration, which is your file-diff versus final-response finding in LLM terms. Dense and context-dependent beats one terminal score, because a terminal score is the easiest thing to shortcut. Build the eval the way an adversary would, then hold out a slice the policy never sees and watch judge-versus-human agreement on it. And I'd say plainly I haven't run this at RLVR scale, so I'd want to see how your in-sandbox verifiers and rubric criteria are calibrated.

2.7 First 90 days (45 s)

> First, own one benchmark or environment slice end to end: run failure analysis on current outputs, categorize and quantify the failure modes, turn the top ones into verifier or data fixes. Second, reproduce the public 35B recipe on your infra so I understand the trainer and harness path before touching it. Third, propose one experiment that tests whether a data or grading change moves Pass@1 more than an algorithm change, because your own ablations say data dominates.

2.8 Logistics (only if asked)

> Yes to San Francisco, five days in person. Canadian citizen, TN-eligible, no sponsorship needed.

Then the start-date line you chose in §4.1. Do not improvise it.

2.9 Your three questions (pick in T9; defaults below)

1. "In the 397B write-up, harness fixes alone moved the 35B model about six points before any RL. How much of the team's time goes into harness and verifier correctness versus the training loop now?"
2. "You found file-diff grading was much harder to overfit than final-response grading. Is that becoming a design rule for new APEX tasks, and who owns that call?"
3. "There are several sibling reqs on the board, Benchmarking, Real Environments, Environments/Data/Post-Training. How does the Post Training req split from those, and what does a normal week look like, Saturdays included?"

Backup if he is on the SkyRL work: "What surprised you most in the de-risking phase before the 397B run?"

2.10 Close (10 s)

> Thanks, this was useful. I'm keen. What's the next step on your side and roughly when?

---

3. Study units

Rules for every unit: read the source in one sitting, write the answers to the self-check **from memory, on paper, after closing the tab**, then check against the key. Anything you miss, re-read that part only, and redo that question the next day. "Passed" means every answer matches the key in substance.

3.1 Unit A: the 397B SkyRL write-up (Sat, 120 min)

Read exactly:

1. https://www.mercor.com/blog/training-frontier-knowledge-work-agents-a-397b-rl-training-guide-with-skyrl/ (Charlie Ruan, Sumanth Hegde, Eric Tang, Tyler Griggs, Jungyeon Park, Maanas Baraya, Philipp Moritz, Michael Haines, Edward J. Hu, et al., 2026-09-01). Whole post, 60 min. Slow down on: the de-risking steps, the ablation table, the generalization section.
2. https://github.com/Mercor-Intelligence/ApexAgents-SkyRL-Recipe README only, 20 min. Skip code.

Learn specifically: what was trained, on what data, how the loop is wired (who does what: SkyRL, Harbor, Modal, vLLM, Megatron), the three de-risking steps and what each caught, the headline numbers, the ablation results and what they imply, the generalization result.

Plain version (read this first, then the source): They took two open Qwen models (a 35B mixture-of-experts and a 397B mixture-of-experts) and trained them with RL on 1,928 expert-written office tasks (consulting, banking, law) from their own APEX-Agents dataset, then tested on 480 held-out tasks. The agent runs inside a sandboxed fake company (documents, email, chat, exposed as MCP tools) and a grader inside the sandbox scores the result; that score is the reward. SkyRL runs the training loop fully asynchronously: rollouts keep generating while the trainer updates weights, and new weights are pushed into the inference engines mid-flight. Before training they did three de-risking passes: (1) run the whole training set at RL concurrency and fix every non-model error (timeouts, judge rate limits, MCP client isolation) until the error rate is near zero; (2) compare per-token log-probabilities between the trainer and the inference engine, which caught a real correctness bug in the vLLM CPU-offload + GDN + in-flight-update combination (they treat a gap under 0.03 as healthy); (3) overfit a handful of tasks first, which showed that tasks graded by diffing files were much harder to overfit than tasks graded on the final chat response, exposing grading weaknesses. Results: 397B went from 16.11% to 27.29% Pass@1; 35B went base 22.74% to 28.69% from harness fixes alone with zero training, then RL added another 10 to 12 points. Ablations on the 35B: averaging the loss per prompt group (prompt_mean) instead of over one global token pool (token_mean) gave +3.9 points because long trajectories were dominating gradients; injecting a "wrap up" notice at 20% context remaining gave +3.0 because fewer rollouts blew the context and got zeroed; the two loss variants (DPPO vs the GLM-5 loss) were within noise; overlong filtering and length penalties were neutral or negative. Their stated lesson: algorithm knobs mattered less than data and harness; the best single knob was +3.9 while post-training overall moved 10 to 12 points. The trained models also improved on a different harness (OpenCode) and on Terminal-Bench 2.1, more so for the small model, and did not regress on HLE/GPQA.

Self-check A (answer from memory, then compare):

1. Which two base models, and how many training tasks vs held-out eval tasks?
2. Name the three de-risking steps in order and what each one caught.
3. What number do they treat as a healthy trainer-vs-inference logprob gap?

4. What is the 397B Pass@1 before and after?
5. On the 35B, how much came from harness fixes alone, and how much from RL?
6. prompt_mean vs token_mean: which won, by how much, and why?
7. What is the "context nudge" and why did it help?
8. Which knobs did nothing or hurt?
9. Which grading style was harder to overfit, and what does that imply for task design?
10. What is the one-sentence lesson about algorithms vs data?
11. In one sentence each: what do SkyRL, Harbor, and Modal do in this setup?

Answer key A: 1. Qwen3.6-35B-A3B and Qwen3.5-397B-A17B; 1,928 train, 480 held-out. 2. Zero the non-model error rate at RL concurrency (timeouts, judge rate limits via round-robin keys, per-process MCP clients); trainer-vs-inference logprob comparison (caught the vLLM CPU-offload + GDN + in-flight-update bug); overfit a handful of tasks (exposed grading weakness: file-diff vs final-response). 3. Below 0.03. 4. 16.11% to 27.29%. 5. Harness fixes: 22.74% to 28.69% with zero training; RL: roughly +10 to 12 points on top. 6. prompt_mean won by 3.9 points; token_mean let 128k-token trajectories dominate the gradient over 2k ones. 7. At 20% context budget remaining, inject a notice to wrap up; fewer rollouts blow the context, fewer get zeroed, more usable signal per batch; +3.0. 8. DPPO vs GLM-5 loss within noise; overlong filtering and adaptive length penalty neutral or negative. 9. File-diff grading; grade the state of the work product, not the model's summary of it. 10. Algorithm choices mattered less than the data: best knob +3.9, post-training as a whole +10 to 12. 11. SkyRL: the async training loop, vLLM engines, in-flight weight sync. Harbor: task/trial lifecycle and running the verifier in-sandbox. Modal: the sandboxes each trial runs in, booted from a prebuilt Docker image.

Output: six sentences in your own words: what they trained, on what, the three de-risking steps, the headline numbers, the ablation lesson, and where it rhymes with your LIBERO-PRO work. Keep it in your notes; this is the raw material for §2.2.

3.2 Unit B: RL post-training concepts, conversation depth (Sun, 90 min)

You are not learning to implement these. You are learning enough to follow and contribute to a 5-minute technical exchange without bluffing.

B1. RLVR (15 min)

- **Read:** the RLVR section of "Tülu 3: Pushing Frontiers in Open Language Model Post-Training", https://arxiv.org/abs/2411.15124 (search the paper for "Verifiable Rewards"; 10 min).
- **Plain version:** RL with Verifiable Rewards means the reward comes from a deterministic checker, not from a learned reward model. Did the math answer match? Did the unit tests pass? Did the file end up in the right state? Reward is typically binary or per-criterion. It avoids the reward-model failure mode where the policy learns to please the model instead of doing the task, but it moves the whole problem onto the verifier: if the verifier can be shortcut, RL will find the shortcut. Mercor's business is building verifiable environments for real work, which is why "verifier hard to game" appears in every one of their JDs.
- **Self-check:** (1) What replaces the reward model in RLVR? (2) Where does the risk move to? (3) Why is this Mercor's whole thesis?
- **Key:** (1) A programmatic or rubric verifier that checks the outcome. (2) Onto the verifier and task design: any exploitable grading gets exploited. (3) Their product is expert-built tasks plus verifiers; "the economy will become an RL environment machine."

B2. PPO to GRPO (25 min)

- **Read:** DeepSeekMath, https://arxiv.org/abs/2402.03300, Section 4.1 only ("From PPO to GRPO"), 20 min. Look at the figure comparing PPO and GRPO.

- **Plain version:** PPO needs a second model (the critic / value model) roughly the size of the policy to estimate how good a state is; that is expensive. GRPO drops the critic. For each prompt it samples a group of G answers, scores them all, and uses the group's mean reward as the baseline: each answer's advantage is (its reward minus the group mean) divided by the group standard deviation. Answers better than their siblings get pushed up, worse ones pushed down. The KL penalty against the reference model is added straight into the loss (with an unbiased estimator) instead of being mixed into the reward. Same clipped-ratio idea as PPO for stability.
- **Self-check:** (1) What does GRPO remove versus PPO and why? (2) Write the outcome-supervision advantage formula. (3) Where does the KL term go? (4) What is "group" in the name?
- **Key:** (1) The value/critic model; it is a comparable-size model and a memory and compute burden. (2) A_i = (r_i − mean(r)) / std(r) over the G samples for the same prompt, applied to every token of output i. (3) Added directly to the loss against the reference policy, not into the reward. (4) The G sampled outputs for one prompt; their scores are the baseline.

B3. DAPO (20 min)

- **Read:** DAPO, https://arxiv.org/abs/2503.14476, Section 3 only (the four techniques), 15 min.
- **Plain version:** DAPO is GRPO-style RL with four fixes that ByteDance Seed found necessary at scale. Clip-Higher: use a bigger upper clip range than lower so low-probability tokens can grow and the policy does not collapse to low entropy. Dynamic Sampling: throw out prompts where every sample is right or every sample is wrong, because they give zero gradient, and keep sampling until the batch is full of informative prompts. Token-Level Policy Gradient Loss: average over tokens rather than per sample so long correct reasoning is not under-weighted. Overlong Reward Shaping: soft length penalty for truncated answers instead of a hard zero. Reward is rule-based: +1 if the final answer matches, −1 otherwise. Result: 50 on AIME 2024 with Qwen2.5-32B base. Note the tension with Unit A: Mercor found token-level averaging (token_mean) *hurt* on agent tasks because trajectories vary from 2k to 128k tokens; prompt_mean won by 3.9. Different regime (multi-hour agent rollouts vs single-shot math), different answer. That contrast is a good thing to say out loud if the topic comes up.
- **Self-check:** (1) Name the four techniques. (2) Why filter prompts with accuracy 0 or 1? (3) What did DAPO say about token-level loss, and what did Mercor find, and why the difference?
- **Key:** (1) Clip-Higher, Dynamic Sampling, Token-Level Policy Gradient Loss, Overlong Reward Shaping. (2) All-same rewards give zero advantage, so no gradient; they waste the batch. (3) DAPO: token-level so long sequences influence the gradient more. Mercor: token_mean let very long agent trajectories dominate; prompt_mean (+3.9) removed the length bias. Regime difference: math answers of similar length vs agent trajectories spanning 2k to 128k tokens.

B4. What "DPPO" and "GLM-5 loss" mean in Mercor's post (10 min)

- **Read:** the ablation section of the 397B post again, just the paragraph on DPPO vs GLM-5 loss.
- **Plain version (only what the post says; do not go beyond it):** both are ways to use the rollout log-probabilities directly in the loss to correct for two things: the trainer and inference engine disagree slightly on token probabilities, and in fully-async training the rollouts were generated by a slightly stale policy. DPPO masks tokens where training and inference logprobs diverge, using a binary approximation of total-variation divergence. The GLM-5 loss instead truncates the importance ratio. They scored within noise of each other; DPPO consistently pushed the model toward more, shorter turns. If asked what DPPO stands for, say you only know it from their post's description. Do not invent an expansion.
- **Self-check:** (1) What problem are both variants correcting? (2) How does each do it? (3) Which won?
- **Key:** (1) Train/inference logprob mismatch and async policy staleness. (2) DPPO masks divergent tokens (binary TV-divergence approximation); GLM-5 loss truncates the importance ratio. (3) Neither; within noise.

B5. Reward hacking and verifier gaming (20 min)

- **Read:** https://www.mercor.com/blog/agent-eval-systems/ (Brandon Lei, 2026-06-30), whole post, 15 min.
- **Plain version:** A verifier is a script or separate agent that grades a trajectory and its output, returns a score in [0, 1] plus an explanation. Weighted verifiers combine into one reward. Three constraints the post names: verifiers must resist gaming, eval coverage must reflect the real work distribution, and domain expertise is required to set the bar. Common gaming patterns you can name from your own work and from Unit A: (a) grading the narration instead of the state (model says "done", file unchanged); fix: grade the artifact, diff files. (b) A single terminal score the policy can satisfy by a shortcut; fix: dense, per-criterion, context-dependent rewards. (c) Judge drift or judge exploitation when the judge is an LLM; fix: calibrate judge against human labels on a held-out slice, track agreement, rotate or ensemble judges. (d) Train/eval overlap or contamination; fix: held-out worlds, not just held-out tasks. Your own line: VLM judges that score rollouts into dense, context-dependent rewards were designed exactly so a single trick cannot satisfy them.
- **Self-check:** (1) Define verifier per the post. (2) Three constraints. (3) Four gaming patterns with one fix each. (4) Which of these have you personally built?
- **Key:** (1) Separate agent/script grading trajectory and output, score in [0,1] with explanation. (2) Resist gaming; coverage matches real work distribution; domain expertise. (3) As above. (4) Dense context-dependent VLM judges (Merlyn); deterministic eval over synthesized inputs (BMO). Not: LLM-judge calibration at scale. Say so.

Output for Unit B: one page of handwritten notes, the four plain versions in your own words. Not for reference in the call; the writing is the learning.

3.3 Unit C: APEX-Agents, Archipelago, Harbor (Sun, 30 min)

- **Read:** https://www.mercor.com/blog/introducing-apex-agents/ (2026-01-21), 15 min. Then the abstract only of https://arxiv.org/abs/2601.14242, 5 min.
- **Plain version:** APEX-Agents is Mercor's benchmark of long, multi-application office tasks in investment banking, consulting, and corporate law. Built by surveying hundreds of practitioners, having experts stage realistic scenarios in Google-Workspace-style datarooms (with Box), and writing 1 to 10 pass/fail criteria per task that define "client-ready." Frontier models complete under 25% of tasks on the first try; about 40% with eight attempts. Archipelago is their open-source harness for running and grading agents in these environments; Harbor manages the trial lifecycle and runs the verifier in the sandbox; the dataset is on Hugging Face under CC-BY. The 397B run trained on 1,928 of these tasks.
- **Self-check:** (1) Three domains. (2) How is a task graded? (3) Frontier Pass@1 at launch? (4) What is Archipelago vs Harbor?
- **Key:** (1) Investment banking, management consulting, corporate law. (2) 1 to 10 expert-written pass/fail criteria per task. (3) Under 25%; ~40% at 8 attempts. (4) Archipelago: harness for executing and evaluating agent trajectories in the environments. Harbor: trial/task lifecycle manager that runs the verifier in-sandbox and returns the reward.

---

4. Worksheets

4.1 Decisions (T1, tonight, 25 min). Write the answers into this file.

D1. Start date and the M.Eng. cv.md says M.Eng expected April 2027. Aksh's team is SF, five days in person. Decide which of these is true and write the exact sentence you will say:

- (a) You would finish the M.Eng remotely or part-time and can start within weeks of an offer: "I can start within 2 to 4 weeks of an offer. My M.Eng is course-based and I'd finish it remotely alongside."
- (b) You would pause or leave the M.Eng for this: "I can start within 2 to 4 weeks of an offer; the degree is course-based and I'd put it on hold for this."

- (c) You cannot start before a certain month: "I can start in [month]." (Weakest; only if true.) Write: D1 = _____

D2. BMO proprietary boundary. Tick what you can say out loud:

- ☐ The $200B+ AUM figure (already on your CV; yes)
- ☐ "Systematic bias to downplay investment risk" (on your CV; yes)
- ☐ The number of synthesized inputs beyond "hundreds"
- ☐ What the tool does for advisors, in general terms
- ☐ Anything about the RL environments beyond "for wealth-management agents"
- ☐ The graph-based system's data sources beyond "client data" Your stop line, verbatim: "That's as far as I can go on the internal detail." Write: D2 = _____

D3. AI Notetaker. Recommendation: leave it on. Opting out is allowed and stated as no impact, but it makes you the exception in a 20-minute slot and gains nothing. Write: D3 = on / off

D4. Other processes. If asked "where else are you interviewing," the honest short form. Write one sentence. Do not list companies. D4 = _____

D5. BMO offer and TMX letter. What did you do on the BMO deadline (09-03 4 PM)? This changes D4 and the timeline answer. Write: D5 = _____

D6. Which CV did Mercor get (~08-20)? If it differs from cv.md, list any bullet on it that is not in §2 so we can reconcile. D6 = _____

4.2 Take-home recall (T3, Fri, 60 min)

Open your Litmus submission. Do not skim; read it as the grader. Then write:

1. **The problem, 5 lines.** What was given, what was asked, what "done" looked like.
2. **Decision 1** (the first real choice you made): what you chose, the alternative you rejected, why. Two sentences.
3. **Decision 2**: same format.
4. **Decision 3**: same format.
5. **What you deliberately skipped** and why (time, scope, or judgment). One sentence each.
6. **With one more day** you would: three bullets, most valuable first.
7. **The weakest part** of your submission, named honestly, and what you would say if he pokes it. Two sentences.
8. **What the grader was looking for**, your best guess in one sentence, and whether you gave it to them.

Then say items 1, 2, 5, 6 out loud, once, timed. Target: 90 seconds total. That is your answer to "tell me about the take-home." Add the 5-line summary from item 1 to the intel doc under Open Item 2.

4.3 Stories fill-in (T7, Mon, 30 min)

Fill Situation (S) and Reflection (R) for each. Task (T), Action (A), Result (Res) are already fixed by cv.md and must not change.

Story 1: LIBERO-PRO recipe

- S: why you looked at π0.5 on LIBERO-PRO in the first place, two sentences. _____
- T: explain the 96% → 21% position-swap collapse.
- A: conservative FFT (batch 64, LR 1e-5), stable 8k–27k steps; LoRA 15–21%; frozen video prior 42→35; batch 16 / LR 1e-6 → 26%.
- Res: 42%, matched standard LIBERO; recipe-induced, not architectural.
- R: what you would do differently, and what you took from the CoRL rejection. _____

Story 2: BMO bias

- S: how the tool came to you (within D2). _____
- T: detect misaligned outputs at scale.
- A: hundreds of synthesized inputs, deterministic pipeline, quantified skew.
- Res: systematic risk-downplaying bias surfaced; now detectable at scale.
- R: what generalizes to any eval pipeline. _____

Story 3: BEHAVIOR-1K

- S: 3-person team, compute-limited, trained on 22 of 50 task types, scored on all 50.
- T: place.
- A: 60% proprioception masking; chunked execution over temporal ensembling (3x); boundary resampling for long-tail subtasks (2x).
- Res: 8th; technical report; LessWrong post (model-organism view of these findings; not the LIBERO-PRO work).
- R: proprioceptive collapse as shortcut learning; what it says about VLA evals. _____

Done-check: read each story aloud in under 2 minutes with the Reflection included.

---

5. Numbers card (T8, Mon, 20 min)

Method: cover the right column, recite, uncover, mark. Three passes. Pass = 100% on the third.

Cue	Number
Wealth-management tool AUM	$200B+
Eval pipeline inputs	hundreds of synthesized inputs
BEHAVIOR-1K placement	8th, Standard Track
BEHAVIOR task types trained / scored	22 trained / 50 scored
BEHAVIOR demonstrations	10,000+
Proprioception masking	60% masked → up to +48% task success
Chunked vs temporal ensembling	3x
Long-tail subtasks via boundary resampling	doubled
π0.5 standard LIBERO → LIBERO-PRO position-swap	96% → 21%
Conservative FFT recipe	batch 64, LR 1e-5; stable 8k–27k steps
Conservative FFT result	21% → 42%
LoRA at matched hparams	15–21%
Frozen video-diffusion prior	42% → 35%
Too-conservative recipe (batch 16, LR 1e-6)	26%
Merlyn Labs	3 people, self-funded, nights and weekends
M.Eng	UofT, AI & Robotics, expected April 2027
Mercor 397B Pass@1	16.11% → 27.29%

Cue	Number
Mercor 35B harness-only	22.74% → 28.69%
Mercor best single knob	prompt_mean +3.9; nudge +3.0
Mercor train / eval tasks	1,928 / 480
Mercor research-eng band (public sibling reqs)	$180K–$500K + equity

---

6. Practice session (T10, Tue, 60 min)

Run it in this repo. Exact invocation, paste as one message:

```
/career-ops interview/practice
Round type: screening/HM, but run it as a technical background conversation with an engineer (20
minutes).
Interviewer persona: James Moore, research engineer on Aksh Garg's research-eng team at Mercor;
likely on the SkyRL 397B post-training work.
Prep file: interview-prep/mercor-research-engineer-post-training.md. Scripts: interview-
prep/mercor-execution-plan.md section 2.
Questions, in this order, one at a time, with one follow-up each:
1. Walk me through your background.
2. Tell me about the take-home: how did you approach it and what would you change?
3. What's your hands-on experience with RL post-training for LLMs?
4. Tell me about a hard technical trade-off you made.
5. How would you design a verifier that's hard to game?
6. What did you take from our 397B write-up?
7. What would your first 90 days here look like?
8. Are you open to SF five days a week, and when could you start?
9. What questions do you have for me?
Enforce the 90-second rule per answer. Score each ✅/🟡/🔴. Write the session summary and
transcript to interview-prep/sessions/.
```

Answer **out loud**, then type what you said (not a cleaned-up version). Record the session on your phone too. After the summary:

- Any 🔴: re-run that single question the same evening until 🟡 or better.
- Any answer over 90 seconds: cut it to the §2 script and re-run once.
- Any claim the session flags as not in cv.md: drop it. Do not argue with the flag.

Done-check: session summary saved; no 🔴 remaining; every answer under 90 seconds on the recording.

T11, Google Meet test (15 min): join https://meet.google.com/gzy-xivr-vyu from the laptop you will use, at the desk you will use, at 13:00 on Tuesday (same light as the real slot). Check camera framing (eyes at upper third), headset audio, background. Write the dial-in on paper: +1 724-565-4894, PIN 766693142. Close the tab.

---

7. Day-of run sheet, Wed 09-09

Time (ET)	Do
08:00–09:00	Normal morning. No new reading.
11:00–11:30	Read §2 scripts once, silently. Read §5 numbers card once. Stop.
11:30–12:30	Nothing Mercor. Eat. Walk.
12:30	Laptop on, charger in, phone silent, Meet link open in a fresh window, notes on paper beside the keyboard: the three questions (§2.9), the numbers card, D1 sentence, D2 stop line. Water.
12:45	Read §8 below, once.
12:57	Join the call. Camera on.
13:00	Open: "Hi James, good to meet you." If he asks how you are: one sentence, then "happy to start wherever you'd like."
13:00–13:02	If he opens with "tell me about yourself": §2.1, stop at 90 s. If he opens with the take-home: §4.2 items 1, 2, 5, 6.
13:02–13:14	His questions. Headline first, then reasons. If you do not know something: "I don't know that; here's what I'd check." Never fill silence. If he goes to LLM RL depth: §2.3 first, then Unit B vocabulary, and stop where your knowledge stops.
13:14	If he has not offered, say: "Can I ask you a couple of things about the team?" Ask §2.9 questions 1 and 3; question 2 only if there is time.
13:18	Close with §2.10.
13:20–13:50	Immediately: write down every question he asked, in his words, and what you answered. Then run `/career-ops interview/debrief` in this repo and paste them. That creates the question bank for the next round.

8. 15-Minute Pre-Interview Review

Anchor sentence: I find where models actually fail and build the evaluation or training fix that closes the gap: bias in a $200B AUM GenAI tool, proprioceptive collapse in VLAs, a published VLA baseline that turned out to be a recipe artifact.

Three things to leave him with:

1. Rigorous failure analysis and reward/verifier design, already done for real, with exact numbers.
2. I read the 397B write-up and understood it: harness fixes gave six points before any RL; file-diff grading resisted overfitting; data beat algorithm knobs. That is my LIBERO-PRO lesson in their domain.
3. The gap is LLM-scale RL infrastructure, said plainly, and it is learnable.

Do not: claim LLM post-training runs; say "published" or "under review" (say "we wrote a paper; it was rejected at CoRL; the result stands"); mention Charlie; raise comp; mention the greeter robot; say "US-authorized"; conflate LessWrong (BEHAVIOR) with the LIBERO-PRO paper.

Your D1 sentence: _____ (from §4.1)

Your questions:

1. Harness and verifier correctness vs the training loop: where does the team's time go now?
2. Is file-diff grading becoming a design rule for new APEX tasks, and who owns that call?
3. How does the Post Training req split from the sibling reqs, and what does a normal week look like, Saturdays included?